Этот урок поможет ребятам превратить разрозненные знания о Flask и SQL в полноценное приложение — **мессенджер с поддержкой личных чатов и передачи файлов**.

Поскольку основы ребятам уже знакомы, мы сфокусируемся на логике **архитектуры данных** и **обработке файлов**.

Урок: Создаем свой защищенный мессенджер на Flask

Сегодня мы разберем, как работает настоящий чат «под капотом». Мы научим наше приложение не просто сохранять текст, но и разделять сообщения на «публичные» и «личные», а также работать с файлами.

1. Проектируем базу данных: Кто и кому пишет?

Раньше мы просто хранили список дел или пользователей. В мессенджере нам нужна таблица `messages`, которая связывает отправителя и получателя.

Разбираем структуру:

- `from_user`: Кто отправил?
- `to_user`: Кому отправили? (Если там пусто/NULL — это общее сообщение в главный чат).
- `file_path`: Путь к картинке или документу на сервере.
- `is_read`: Маленький флажок (0 или 1), чтобы знать, видел ли пользователь сообщение.

Задание: Найдите в функции `init_db()` блок с `ALTER TABLE`. Зачем он нужен? (Ответ: чтобы «достроить» таблицу, если мы решили добавить новые функции, не удаляя старые данные).

---

2. Логика чатов: Как не подсматривать чужие письма?

Самая важная часть кода находится в функции `index()`. Когда пользователь открывает страницу, сервер должен решить, какие сообщения ему показать.

- **Общий чат:** Если `active_chat` не выбран, мы пишем SQL-запрос: `SELECT * FROM messages WHERE to_user IS NULL`.
- **Личный чат:** Если мы нажали на пользователя «Никита», нам нужны сообщения, где:
    1. Отправитель — я, а получатель — Никита.
    2. **ИЛИ** Отправитель — Никита, а получатель — я.

SQL-магия:

SQL
```
SELECT * FROM messages
WHERE (from_user = ? AND to_user = ?) OR (from_user = ? AND to_user = ?)
```

---

3. Работа с файлами: Безопасность прежде всего

Когда мы принимаем файл от пользователя, нельзя просто сохранить его под старым именем. Злоумышленник может назвать файл `../../config.py` и перезаписать настройки нашего сервера!

Как мы это решаем в коде:

1. `allowed_file`: Проверяем расширение (разрешаем только картинки и документы).
2. `secure_filename`: Функция из библиотеки `werkzeug`, которая убирает из имени файла все опасные символы.
3. `os.makedirs`: Создаем папку `uploads`, если её ещё нет.

---

4. Функция уведомлений (Check for updates)

Чтобы мессенджер был «живым», он должен проверять новые сообщения без перезагрузки страницы. Для этого в коде есть маршрут `/check_new_messages`.

Он возвращает **JSON** — формат данных, который легко понимает язык JavaScript в браузере. Если в базе появилось сообщение, где `is_read = 0`, сервер отправит сигнал: «Эй, у тебя новое сообщение от пользователя X!».

Практические задания для учеников:

1. **Смена пароля:** В коде есть регистрация и вход. Добавьте в базу данных возможность смены пароля в личном кабинете.
2. **Удаление сообщений:** Добавьте кнопку «Удалить», которая будет стирать сообщение из базы данных (но только если его удаляет сам автор!).
3. **Статус "В сети":** Попробуйте придумать, как можно хранить информацию о том, что пользователь сейчас открыл сайт.
4. **Улучшение безопасности:** Сейчас пароли хранятся в открытом виде (простым текстом). Это очень плохо! Погуглите, как использовать библиотеку `werkzeug.security` для создания хешей паролей.

Как запустить проект?

Для работы мессенджера нужны сертификаты безопасности (так как используется `ssl_context`).

1. Убедитесь, что у вас есть файлы `cert.pem` и `key.pem`.
2. Запустите `main.py`.
3. Зайдите в браузер по адресу `https://0.0.0.0:5050` (не забудьте про **https**!).

Этот проект — отличная база для создания своего крутого социального сервиса!

В нашем проекте используются две таблицы: `users` (кто пользуется) и `messages` (кто, что и кому пишет).

---

1. Таблица `users`: Регистрация пользователей

Это простая таблица для авторизации. Она хранит «паспортные данные» каждого участника.

Поле (Column)	Тип данных	Описание
id	INTEGER	Уникальный номер (ID) пользователя. Устанавливается автоматически.
username	TEXT	Уникальный логин. В коде стоит `UNIQUE`, чтобы не было двух «Админов».
password	TEXT	Пароль пользователя.

---

2. Таблица `messages`: Сердце мессенджера

Эта таблица сложнее, так как она должна поддерживать и общие сообщения, и приватные диалоги, и даже пересылку файлов.

Разбор полей:

- **from_user**: Здесь хранится `username` того, кто нажал кнопку «Отправить».
- **to_user**: Самое интересное поле.
  - Если там стоит имя (например, `Ivan`), значит это **приватное сообщение**.
  - Если там пусто (`NULL`), значит сообщение **публичное** и его видят все в общем чате.
- **message**: Текст самого сообщения.
- **file_path**: Если ребенок прикрепил картинку, программа сохранит её в папку `static/uploads`, а в это поле запишет путь, например: `uploads/my_cat.jpg`. Если файла нет, поле остается пустым.
- **timestamp**: Время отправки. Заполняется автоматически сервером базы данных (`DEFAULT CURRENT_TIMESTAMP`).
- **is_read**: Числовое поле (булево).
  - `0` — сообщение новое.
  - `1` — сообщение прочитано. Это позволяет нам рисовать «галочки» или показывать уведомления в браузере.

---

3. Как работают запросы (SQL)

Чтобы дети поняли механику, можно показать им разницу в логике запросов:

А) "Кто мне написал?" (Уведомления)

Программа ищет все записи, где получатель — текущий пользователь, а статус — «не прочитано»:

SQL
```
SELECT from_user FROM messages
WHERE to_user = 'МоёИмя' AND is_read = 0;
```

Б) "Покажи нашу переписку"

Когда мы открываем чат с другом, нам нужно достать сообщения в обе стороны (мои ему и его мне), отсортированные по времени:

SQL
```
SELECT * FROM messages
WHERE (from_user = 'Я' AND to_user = 'Друг')
   OR (from_user = 'Друг' AND to_user = 'Я')
ORDER BY timestamp ASC;
```

Совет для урока:

Предложите ребятам открыть файл `database.db` через любой просмотрщик SQLite (например, **DB Browser for SQLite**). Когда они увидят, как их сообщения в реальном времени появляются в строках таблицы, магия кода превратится в понятную инженерную конструкцию.

Маршруты

Давай разберем, как Flask превращает действия пользователя в браузере в те самые SQL-запросы, о которых мы говорили.

В коде за это отвечает функция **`index()`**. Это "диспетчер", который обрабатывает и отображение сообщений (GET-запрос), и их отправку (POST-запрос).

---

1. Отправка сообщения (POST)

Когда ребенок нажимает кнопку «Отправить», браузер посылает данные на сервер. Flask ловит их через `request.form`.

Логика на сервере:

1. **Кто пишет?** Имя берется из сессии: `username = session['username']`.

2. **Кому?** Если в поле `to_user` пусто, Flask запишет `None` (в базе это превратится в `NULL`).
3. **Файл:** Если прикреплен файл, сервер сохраняет его в папку и запоминает путь.
4. **Запись:** Выполняется команда `INSERT INTO messages`.

Python
```
# Упрощенная логика из кода:
c.execute("INSERT INTO messages (from_user, to_user, message, file_path)
VALUES (?, ?, ?, ?)",
          (username, to_user, message, file_path))
```

---

2. Получение сообщений (GET)

Когда мы просто открываем страницу, Flask должен подготовить список сообщений. Тут работает логика разделения чатов:

- **Если открыт профиль друга (`active_chat` существует):** Сервер делает два дела сразу:
    1. **Отмечает сообщения как прочитанные:** `UPDATE messages SET is_read = 1 WHERE to_user = ? AND from_user = ?`. Теперь отправитель увидит, что его сообщение открыли.
    2. **Загружает историю:** Достает только те строки, где участвуют эти два пользователя.
- **Если открыт общий чат:** Сервер достает всё, где `to_user IS NULL`.

---

3. Механизм "Живого чата" (AJAX)

Самая интересная функция в коде — `check_new_messages`. Она работает незаметно для пользователя.

Как это объяснить детям: Представьте, что браузер каждые несколько секунд "шепотом" спрашивает у сервера: *"Есть что-нибудь новенькое для меня?"*.

1. Браузер обращается к маршруту `/check_new_messages`.
2. Flask выполняет быстрый запрос: `SELECT from_user FROM messages WHERE to_user = ? AND is_read = 0`.
3. Сервер отвечает списком имен (JSON). Например: `{"new_messages_from": ["Ivan", "Mary"]}`.
4. JavaScript на странице видит этот список и подсвечивает нужные чаты или выводит уведомление.

---

4. Роль сессий (`session`)

Чтобы сервер не спрашивал пароль при каждом клике, используется `session['username']`.

- При **Login** мы записываем имя в "куки" браузера.

- При каждом действии Flask проверяет: "А этот пользователь вообще залогинен?".
- При **Logout** мы просто удаляем имя из сессии: `session.pop('username', None)`.

Идея для практического мини-задания:

Попросите ребят изменить SQL-запрос в `index()` так, чтобы сообщения в общем чате показывались не все сразу, а только за последние 24 часа. Это научит их работать с полем `timestamp`.

Хочешь, подберем конкретные вопросы для проверки знаний по этой части кода?